



Facultad de Ingeniería
Escuela de Informática

Análisis y Modelamiento Matemático del Problema de Secuenciación de Trabajos (Job Sequencing)

NP-Completo: Secuenciación de Trabajos

Integrantes:

Rodrigo Alejandro Molina Castillo

Tomas Gamonal

Bryan Madrid

Profesor:

Luis Fuentes Larenas

Santiago, Chile
Mayo, 2026

Índice

1. Definición del problema	2
2. Ejemplo ilustrativo	2
2.1. Verificación de la solución	3
2.2. Comparación con otra secuencia	3
2.3. Representación temporal de la solución	3
2.4. Interpretación	4
3. Aplicaciones	4
4. Modelo matemático	4
4.1. Conjuntos	5
4.2. Parámetros	5
4.3. Variables de decisión	5
4.4. Función objetivo	6
4.5. Restricciones	6
4.6. Modelo de programación dinámica utilizado	6
5. Complejidad y contexto	7
6. Implementación	8
6.1. Enfoque exacto: Fuerza Bruta / Backtracking	8
6.2. Enfoque mejorado: Programación Dinámica Exacta	8
6.3. Códigos	9
6.3.1. Código en Python	9
6.3.2. Salida obtenida en Python	13
6.3.3. Código en C++	13
6.3.4. Salida obtenida en C++	16
6.3.5. Código en Java	16
6.3.6. Salida obtenida en Java	18
7. Análisis experimental	19
7.1. Análisis de resultados	19
7.2. Comparación entre enfoques	19
7.3. Número de iteraciones estimadas	20
7.3.1. Fuerza Bruta / Backtracking	20
7.3.2. Programación Dinámica	20
7.4. Comparación directa	21
7.5. Comparación entre lenguajes	21
7.6. Escalabilidad	21
8. Conclusiones	22

1. Definición del problema

El problema estudiado corresponde a *Job Sequencing* con duraciones variables, una variante del problema de planificación de trabajos incluida dentro de los problemas NP-completos clásicos estudiados por Richard Karp.

En este problema se tiene un conjunto de trabajos, donde cada trabajo posee una duración, un *deadline* y una ganancia asociada. El objetivo consiste en seleccionar un subconjunto de trabajos y ordenarlos de manera que cada uno finalice antes o justo en su *deadline*, maximizando la ganancia total obtenida.

A diferencia del *Job Sequencing* clásico con duraciones unitarias, en esta versión cada trabajo puede tener una duración distinta. Esto aumenta la dificultad del problema, ya que no basta con asignar trabajos a espacios de tiempo individuales, sino que se debe verificar si la suma acumulada de duraciones permite cumplir los *deadlines*.

2. Ejemplo ilustrativo

Se considera el siguiente conjunto de trabajos:

Trabajo	Duración (p_j)	Deadline (d_j)	Ganancia (w_j)
A	3	4	100
B	2	1	19
C	5	2	27
D	1	1	25
E	4	3	15
F	2	2	30
G	6	3	50
H	3	4	60
I	7	4	20
J	4	3	45
K	2	4	35
L	5	12	80
M	3	5	40
N	4	8	55
O	1	3	22

Cuadro 1: Conjunto de trabajos utilizado en el ejemplo.

Se dispone de una única máquina que ejecuta los trabajos de manera secuencial, por lo que el orden en que se realizan influye directamente en el resultado. A diferencia del caso clásico, cada trabajo posee una duración variable, por lo que ocupa distintos intervalos de tiempo.

Al evaluar distintas posibles secuencias, se obtiene que la mejor solución es:

$$D \rightarrow A \rightarrow H \rightarrow L$$

2.1. Verificación de la solución

- **Trabajo D:** se ejecuta en el intervalo $[0, 1]$ y cumple su *deadline*:

$$1 \leq 2$$

- **Trabajo A:** se ejecuta en el intervalo $[1, 4]$ y cumple su *deadline*:

$$4 \leq 4$$

- **Trabajo H:** se ejecuta en el intervalo $[4, 7]$ y cumple su *deadline*:

$$7 \leq 10$$

- **Trabajo L:** se ejecuta en el intervalo $[7, 12]$ y cumple su *deadline*:

$$12 \leq 12$$

La ganancia total de la secuencia es:

$$w_D + w_A + w_H + w_L = 25 + 100 + 60 + 80 = 265$$

Por lo tanto, la ganancia máxima obtenida es:

$$W = 265$$

2.2. Comparación con otra secuencia

Inicialmente, podría considerarse una secuencia como:

$$A \rightarrow D \rightarrow F$$

Sin embargo, al evaluar más combinaciones mediante un enfoque exhaustivo, se observa que no todas las combinaciones cumplen las restricciones de tiempo, o bien generan una ganancia menor. Esto evidencia la importancia de analizar distintas configuraciones posibles para encontrar la solución óptima.

2.3. Representación temporal de la solución

Tiempo: 0 1 4 7 12
 |---D---|----A-----|----H-----|-----L-----|
 (0->1) (1->4) (4->7) (7->12)

2.4. Interpretación

La representación temporal permite observar, paso a paso, cómo se construye una secuencia factible de trabajos en una única máquina. En primer lugar, se ejecuta el trabajo D , que posee una duración de 1 unidad de tiempo y finaliza en el instante 1, antes de su deadline $d_D = 2$. Luego, se ejecuta el trabajo A , cuya duración es de 3 unidades de tiempo, ocupando el intervalo $[1, 4]$ y finalizando exactamente en su deadline $d_A = 4$.

Posteriormente, se incorpora el trabajo H , el cual se ejecuta desde el tiempo 4 hasta el tiempo 7. Como su deadline es $d_H = 10$, este trabajo cumple la restricción temporal sin generar atraso. Finalmente, se ejecuta el trabajo L , que ocupa el intervalo $[7, 12]$ y finaliza justo en su deadline $d_L = 12$.

En conjunto, la secuencia $D \rightarrow A \rightarrow H \rightarrow L$ utiliza completamente el horizonte de planificación hasta el tiempo 12, sin que exista superposición entre trabajos. Cada trabajo comienza una vez que el anterior ha terminado, lo que respeta la condición de una única máquina disponible.

Además, esta secuencia alcanza una ganancia total de:

$$25 + 100 + 60 + 80 = 265$$

Por lo tanto, la representación temporal evidencia que la solución seleccionada no solo es factible, sino que también maximiza la ganancia total dentro de las restricciones de duración y deadline del problema.

3. Aplicaciones

El problema tiene múltiples aplicaciones en la industria y la informática. Se utiliza en:

- Planificación de producción, donde las máquinas deben procesar trabajos con fechas límite.
- Gestión de proyectos, considerando tareas con duración variable.
- Sistemas operativos, para la planificación de procesos.
- Optimización logística, en la organización de entregas o rutas.

4. Modelo matemático

Para modelar el problema de *Job Sequencing* con duraciones variables, se considera un conjunto de trabajos que deben ser ejecutados en una única máquina. Cada trabajo posee una duración, un *deadline* y una ganancia asociada. El objetivo es seleccionar y ordenar los trabajos de forma que todos finalicen antes de su fecha límite, maximizando la ganancia total obtenida.

4.1. Conjuntos

Sea:

$$J = \{1, 2, \dots, n\}$$

el conjunto de trabajos disponibles.

4.2. Parámetros

Para cada trabajo $j \in J$:

$$p_j = \text{duración del trabajo } j$$

$$d_j = \text{deadline del trabajo } j$$

$$w_j = \text{ganancia del trabajo } j$$

Además, se define el horizonte de planificación como:

$$H = \max_{j \in J}(d_j)$$

En el caso estudiado:

$$H = 12$$

4.3. Variables de decisión

Se define la variable binaria:

$$x_j = \begin{cases} 1, & \text{si el trabajo } j \text{ es seleccionado} \\ 0, & \text{si el trabajo } j \text{ no es seleccionado} \end{cases}$$

Además, se define:

$$C_j = \text{tiempo de finalización del trabajo } j$$

4.4. Función objetivo

El objetivo es maximizar la ganancia total de los trabajos seleccionados:

$$\text{máx } Z = \sum_{j \in J} w_j x_j$$

4.5. Restricciones

Cada trabajo seleccionado debe finalizar antes o justo en su *deadline*:

$$C_j \leq d_j \quad \forall j \in J \text{ tal que } x_j = 1$$

La suma de las duraciones de los trabajos seleccionados no puede superar el horizonte de planificación:

$$\sum_{j \in J} p_j x_j \leq H$$

Además, los trabajos deben ejecutarse de manera secuencial, sin superposición, ya que se dispone de una única máquina.

Finalmente, la variable de selección es binaria:

$$x_j \in \{0, 1\} \quad \forall j \in J$$

4.6. Modelo de programación dinámica utilizado

Como mejora al enfoque exhaustivo, se utilizó programación dinámica exacta. Para ello, se define:

$$DP[i][t]$$

como la máxima ganancia que se puede obtener considerando los primeros i trabajos y un tiempo acumulado t .

La recurrencia utilizada es:

$$DP[i][t] = \text{máx} (DP[i-1][t], DP[i-1][t-p_i] + w_i)$$

siempre que:

$$p_i \leq t \quad \text{y} \quad t \leq d_i$$

En caso contrario:

$$DP[i][t] = DP[i - 1][t]$$

El valor óptimo se obtiene mediante:

$$\max_{0 \leq t \leq H} DP[n][t]$$

Este modelo permite obtener una solución exacta en tiempo pseudo-polinomial, dependiendo del número de trabajos n y del horizonte de planificación H .

5. Complejidad y contexto

El problema de *Job Sequencing* con duraciones variables pertenece a la clase NP, ya que una solución candidata puede verificarse en tiempo polinomial. Dada una secuencia de trabajos seleccionados, basta con calcular el tiempo acumulado de ejecución y comprobar que cada trabajo finalice antes o justo en su *deadline*.

Formalmente, para cada trabajo seleccionado debe cumplirse:

$$C_j \leq d_j$$

donde C_j representa el tiempo de finalización del trabajo j , y d_j su *deadline*.

Además, esta variante del problema se considera NP-completa debido a su relación con problemas clásicos como *Knapsack*, donde se busca seleccionar un subconjunto de elementos maximizando una ganancia bajo restricciones. En este caso, los trabajos poseen duración, *deadline* y ganancia, por lo que se debe decidir qué subconjunto seleccionar y en qué orden ejecutarlo.

En este trabajo se considera también la versión de decisión del problema, donde se determina si existe una secuencia factible de trabajos cuya ganancia total sea al menos un valor K :

$$\sum_{j \in J} w_j x_j \geq K$$

La implementación mediante fuerza bruta/backtracking muestra que el número de combinaciones crece de manera exponencial, ya que cada trabajo puede ser seleccionado o descartado. Por lo tanto, para n trabajos se pueden generar hasta:

$$2^n$$

subconjuntos posibles.

En el caso estudiado, con $n = 15$, esto corresponde a:

$$2^{15} = 32768$$

subconjuntos posibles, lo que evidencia la complejidad combinatoria del problema.

6. Implementación

Para abordar el problema se implementaron dos enfoques con el objetivo de comparar rendimiento y calidad de solución:

1. un enfoque exacto basado en fuerza bruta/backtracking;
2. un enfoque mejorado basado en programación dinámica exacta.

Ambos enfoques fueron implementados en los lenguajes Python, C++ y Java.

Además, las implementaciones fueron desarrolladas de manera directa, evitando el uso de librerías que facilitaran la resolución del problema. Por ejemplo, no se utilizaron funciones automáticas para generar combinaciones ni funciones de ordenamiento avanzadas; dichas operaciones fueron implementadas manualmente mediante ciclos y estructuras básicas.

6.1. Enfoque exacto: Fuerza Bruta / Backtracking

Se implementó un algoritmo exacto basado en fuerza bruta mediante backtracking, el cual explora todas las posibles combinaciones de trabajos. Para cada trabajo, el algoritmo considera dos decisiones:

- seleccionar el trabajo;
- no seleccionar el trabajo.

Esto genera un árbol de búsqueda de tamaño exponencial:

$$2^n$$

Cada subconjunto generado se verifica ordenando los trabajos por *deadline* y calculando el tiempo acumulado de ejecución. Si en algún punto el tiempo acumulado supera el *deadline* de un trabajo, la solución se considera no factible.

6.2. Enfoque mejorado: Programación Dinámica Exacta

Como solución mejorada se implementó un algoritmo de programación dinámica exacta. Este enfoque reduce el espacio de búsqueda al construir soluciones parciales a partir de subproblemas más pequeños.

Primero, los trabajos se ordenan por *deadline* creciente. Luego, se define una tabla:

$$DP[i][t]$$

donde $DP[i][t]$ representa la máxima ganancia que puede obtenerse considerando los primeros i trabajos y un tiempo acumulado t .

La recurrencia principal corresponde a:

$$DP[i][t] = \text{máx} (DP[i-1][t], DP[i-1][t-p_i] + w_i)$$

cuando:

$$p_i \leq t \leq d_i$$

En caso contrario:

$$DP[i][t] = DP[i-1][t]$$

6.3. Códigos

Los códigos fueron implementados en Python, C++ y Java. Cada implementación incluye la solución base mediante fuerza bruta/backtracking y la solución mejorada mediante programación dinámica exacta. Además, se evitó el uso de librerías que facilitaran directamente la resolución del problema, implementando manualmente la generación de combinaciones, el ordenamiento por deadline y la reconstrucción de la solución.

6.3.1. Código en Python

Listing 1: Implementación en Python del Job Sequencing con duraciones variables

```

1 import time
2
3 class Job:
4     def __init__(self, id, duration, deadline, profit):
5         self.id = id
6         self.duration = duration
7         self.deadline = deadline
8         self.profit = profit
9
10    def __repr__(self):
11        return f"{self.id}(p={self.duration}, d={self.deadline}, w={
12            self.profit})"
13
14 jobs = [
15     Job("A", 3, 4, 100),

```

```

16     Job("B", 2, 3, 19),
17     Job("C", 5, 7, 27),
18     Job("D", 1, 2, 25),
19     Job("E", 4, 6, 15),
20     Job("F", 2, 5, 30),
21     Job("G", 6, 8, 50),
22     Job("H", 3, 10, 60),
23     Job("I", 7, 9, 20),
24     Job("J", 4, 7, 45),
25     Job("K", 2, 4, 35),
26     Job("L", 5, 12, 80),
27     Job("M", 3, 5, 40),
28     Job("N", 4, 8, 55),
29     Job("O", 1, 3, 22)
30 ]
31
32
33 def ordenar_por_deadline(lista):
34     lista = lista[:]
35     n = len(lista)
36
37     for i in range(n):
38         for j in range(0, n - i - 1):
39             if lista[j].deadline > lista[j + 1].deadline:
40                 lista[j], lista[j + 1] = lista[j + 1], lista[j]
41
42     return lista
43
44
45 def is_feasible(subset):
46     subset = ordenar_por_deadline(subset)
47
48     tiempo = 0
49     for job in subset:
50         tiempo += job.duration
51         if tiempo > job.deadline:
52             return False
53
54     return True
55
56
57 def calcular_ganancia(subset):
58     total = 0
59     for job in subset:
60         total += job.profit
61     return total
62
63

```

```

64 def brute_force_backtracking(jobs):
65     mejor = {
66         "profit": 0,
67         "subset": []
68     }
69
70     def backtrack(i, actual):
71         if i == len(jobs):
72             if is_feasible(actual):
73                 ganancia = calcular_ganancia(actual)
74                 if ganancia > mejor["profit"]:
75                     mejor["profit"] = ganancia
76                     mejor["subset"] = actual[:]
77             return
78
79         # No tomar el trabajo i
80         backtrack(i + 1, actual)
81
82         # Tomar el trabajo i
83         actual.append(jobs[i])
84         backtrack(i + 1, actual)
85         actual.pop()
86
87     backtrack(0, [])
88
89     mejor["subset"] = ordenar_por_deadline(mejor["subset"])
90     return mejor["subset"], mejor["profit"]
91
92
93 def dp_exacta(jobs):
94     trabajos = ordenar_por_deadline(jobs)
95
96     n = len(trabajos)
97
98     H = 0
99     for job in trabajos:
100         if job.deadline > H:
101             H = job.deadline
102
103     dp = []
104     tomar = []
105
106     for i in range(n + 1):
107         fila_dp = []
108         fila_tomar = []
109         for t in range(H + 1):
110             fila_dp.append(0)
111             fila_tomar.append(False)

```

```

112         dp.append(fila_dp)
113         tomar.append(fila_tomar)
114
115     for i in range(1, n + 1):
116         job = trabajos[i - 1]
117
118         for t in range(H + 1):
119             dp[i][t] = dp[i - 1][t]
120
121             if job.duration <= t and t <= job.deadline:
122                 valor = dp[i - 1][t - job.duration] + job.profit
123
124                 if valor > dp[i][t]:
125                     dp[i][t] = valor
126                     tomar[i][t] = True
127
128     best_time = 0
129     best_profit = 0
130
131     for t in range(H + 1):
132         if dp[n][t] > best_profit:
133             best_profit = dp[n][t]
134             best_time = t
135
136     seleccion = []
137     t = best_time
138
139     for i in range(n, 0, -1):
140         if tomar[i][t]:
141             job = trabajos[i - 1]
142             seleccion.append(job)
143             t -= job.duration
144
145     seleccion = seleccion[::-1]
146     return seleccion, best_profit
147
148
149 if __name__ == "__main__":
150     print("===_JOB_SEQUENCING_CRUDO_-_PYTHON===")
151
152     inicio = time.perf_counter()
153     sol_fb, val_fb = brute_force_backtracking(jobs)
154     t_fb = (time.perf_counter() - inicio) * 1000
155
156     inicio = time.perf_counter()
157     sol_dp, val_dp = dp_exacta(jobs)
158     t_dp = (time.perf_counter() - inicio) * 1000
159

```

```

160     print("\nFuerza Bruta / Backtracking:")
161     print("Seleccion:", sol_fb)
162     print("Ganancia:", val_fb)
163     print(f"Tiempo: {t_fb:.6f}ms")
164
165     print("\nDP Exacta:")
166     print("Seleccion:", sol_dp)
167     print("Ganancia:", val_dp)
168     print(f"Tiempo: {t_dp:.6f}ms")

```

6.3.2. Salida obtenida en Python

Listing 2: Salida obtenida en Python

```

1 Fuerza Bruta / Backtracking:
2 Seleccion: [D(p=1, d=2, w=25), A(p=3, d=4, w=100), H(p=3, d=10, w=60)
   , L(p=5, d=12, w=80)]
3 Ganancia : 265
4 Tiempo   : 203.925500 ms
5
6 DP Exacta:
7 Seleccion: [D(p=1, d=2, w=25), A(p=3, d=4, w=100), H(p=3, d=10, w=60)
   , L(p=5, d=12, w=80)]
8 Ganancia : 265
9 Tiempo   : 0.106200 ms

```

6.3.3. Código en C++

Para mantener el informe legible, el código C++ se presenta como implementación equivalente al caso Python, utilizando arreglos estáticos, identificadores tipo `char`, ordenamiento manual por deadline y medición de tiempo con `chrono`.

Listing 3: Estructura principal de la implementación en C++

```

1 #include <iostream>
2 #include <chrono>
3
4 using namespace std;
5
6 struct Job {
7     char id;
8     int duration;
9     int deadline;
10    int profit;
11 };
12
13 const int N = 15;
14 const int MAX_H = 100;

```

```

15
16 Job jobs[N] = {
17     {'A', 3, 4, 100},
18     {'B', 2, 3, 19},
19     {'C', 5, 7, 27},
20     {'D', 1, 2, 25},
21     {'E', 4, 6, 15},
22     {'F', 2, 5, 30},
23     {'G', 6, 8, 50},
24     {'H', 3, 10, 60},
25     {'I', 7, 9, 20},
26     {'J', 4, 7, 45},
27     {'K', 2, 4, 35},
28     {'L', 5, 12, 80},
29     {'M', 3, 5, 40},
30     {'N', 4, 8, 55},
31     {'O', 1, 3, 22}
32 };

```

Listing 4: Funciones principales de C++: factibilidad, backtracking y DP

```

1 void ordenarPorDeadline(Job arr[], int n) {
2     for (int i = 0; i < n - 1; i++) {
3         for (int j = 0; j < n - i - 1; j++) {
4             if (arr[j].deadline > arr[j + 1].deadline) {
5                 Job temp = arr[j];
6                 arr[j] = arr[j + 1];
7                 arr[j + 1] = temp;
8             }
9         }
10    }
11 }
12
13 bool isFeasible(Job subset[], int size) {
14     Job copia[N];
15
16     for (int i = 0; i < size; i++) {
17         copia[i] = subset[i];
18     }
19
20     ordenarPorDeadline(copia, size);
21
22     int tiempo = 0;
23
24     for (int i = 0; i < size; i++) {
25         tiempo += copia[i].duration;
26
27         if (tiempo > copia[i].deadline) {

```

```

28         return false;
29     }
30 }
31
32     return true;
33 }
34
35 void backtracking(int i) {
36     if (i == N) {
37         if (isFeasible(actual, actualSize)) {
38             int ganancia = calcularGanancia(actual, actualSize);
39
40             if (ganancia > mejorGanancia) {
41                 mejorGanancia = ganancia;
42                 mejorSize = actualSize;
43
44                 for (int k = 0; k < actualSize; k++) {
45                     mejor[k] = actual[k];
46                 }
47             }
48         }
49         return;
50     }
51
52     backtracking(i + 1);
53
54     actual[actualSize] = jobs[i];
55     actualSize++;
56
57     backtracking(i + 1);
58
59     actualSize--;
60 }

```

6.3.4. Salida obtenida en C++

Listing 5: Salida obtenida en C++

```
1 Fuerza Bruta / Backtracking:
2 Seleccion: D(p=1, d=2, w=25) A(p=3, d=4, w=100) H(p=3, d=10, w=60) L(
   p=5, d=12, w=80)
3 Ganancia : 265
4 Tiempo   : 19.999 ms
5
6 DP Exacta:
7 Seleccion: D(p=1, d=2, w=25) A(p=3, d=4, w=100) H(p=3, d=10, w=60) L(
   p=5, d=12, w=80)
8 Ganancia : 265
9 Tiempo   : 0 ms
```

6.3.5. Código en Java

La versión en Java mantiene la misma lógica del algoritmo: arreglos estáticos, ordenamiento manual por deadline, backtracking y programación dinámica exacta.

Listing 6: Estructura principal de la implementación en Java

```
1 public class JobSequencingNP {
2
3     static class Job {
4         String id;
5         int duration;
6         int deadline;
7         int profit;
8
9         Job(String id, int duration, int deadline, int profit) {
10             this.id = id;
11             this.duration = duration;
12             this.deadline = deadline;
13             this.profit = profit;
14         }
15
16         public String toString() {
17             return id + "(p=" + duration + ", d=" + deadline + ", w="
18                 + profit + ")";
19         }
20     }
21
22     static final int N = 15;
23
24     static Job[] jobs = {
25         new Job("A", 3, 4, 100),
26         new Job("B", 2, 3, 19),
```

```

26         new Job("C", 5, 7, 27),
27         new Job("D", 1, 2, 25),
28         new Job("E", 4, 6, 15),
29         new Job("F", 2, 5, 30),
30         new Job("G", 6, 8, 50),
31         new Job("H", 3, 10, 60),
32         new Job("I", 7, 9, 20),
33         new Job("J", 4, 7, 45),
34         new Job("K", 2, 4, 35),
35         new Job("L", 5, 12, 80),
36         new Job("M", 3, 5, 40),
37         new Job("N", 4, 8, 55),
38         new Job("O", 1, 3, 22)
39     };
40 }

```

Listing 7: Funciones principales de Java: ordenamiento, factibilidad y backtracking

```

1  static void ordenarPorDeadline(Job[] arr, int size) {
2      for (int i = 0; i < size - 1; i++) {
3          for (int j = 0; j < size - i - 1; j++) {
4              if (arr[j].deadline > arr[j + 1].deadline) {
5                  Job temp = arr[j];
6                  arr[j] = arr[j + 1];
7                  arr[j + 1] = temp;
8              }
9          }
10     }
11 }

12
13 static boolean isFeasible(Job[] subset, int size) {
14     Job[] copia = new Job[N];
15
16     for (int i = 0; i < size; i++) {
17         copia[i] = subset[i];
18     }
19
20     ordenarPorDeadline(copia, size);
21
22     int tiempo = 0;
23
24     for (int i = 0; i < size; i++) {
25         tiempo += copia[i].duration;
26
27         if (tiempo > copia[i].deadline) {
28             return false;
29         }
30     }

```

```

31
32     return true;
33 }
34
35 static void backtracking(int i) {
36     if (i == N) {
37         if (isFeasible(actual, actualSize)) {
38             int ganancia = calcularGanancia(actual, actualSize);
39
40             if (ganancia > mejorGanancia) {
41                 mejorGanancia = ganancia;
42                 mejorSize = actualSize;
43
44                 for (int k = 0; k < actualSize; k++) {
45                     mejor[k] = actual[k];
46                 }
47             }
48         }
49         return;
50     }
51
52     backtracking(i + 1);
53
54     actual[actualSize] = jobs[i];
55     actualSize++;
56
57     backtracking(i + 1);
58
59     actualSize--;
60 }

```

6.3.6. Salida obtenida en Java

Listing 8: Salida obtenida en Java

```

1 Fuerza Bruta / Backtracking:
2 Seleccion: D(p=1, d=2, w=25) A(p=3, d=4, w=100) H(p=3, d=10, w=60) L(
   p=5, d=12, w=80)
3 Ganancia : 265
4 Tiempo   : 16,787500 ms
5
6 DP Exacta:
7 Seleccion: D(p=1, d=2, w=25) A(p=3, d=4, w=100) H(p=3, d=10, w=60) L(
   p=5, d=12, w=80)
8 Ganancia : 265
9 Tiempo   : 2,616800 ms

```

7. Análisis experimental

Para el análisis experimental se utilizó un conjunto de 15 trabajos con duraciones variables, *deadlines* y ganancias distintas. Los algoritmos fueron ejecutados en Python, C++ y Java.

Lenguaje	Algoritmo	Solución encontrada	Ganancia	Tiempo
Python	Fuerza Bruta / Backtracking	D, A, H, L	265	203,925500 ms
Python	DP Exacta	D, A, H, L	265	0,106200 ms
C++	Fuerza Bruta / Backtracking	D, A, H, L	265	19,999 ms
C++	DP Exacta	D, A, H, L	265	0 ms
Java	Fuerza Bruta / Backtracking	D, A, H, L	265	16,787500 ms
Java	DP Exacta	D, A, H, L	265	2,616800 ms

Cuadro 2: Resultados experimentales por lenguaje y algoritmo.

7.1. Análisis de resultados

Los resultados muestran que ambos enfoques encuentran la misma solución óptima:

$$D \rightarrow A \rightarrow H \rightarrow L, \quad W = 265$$

Sin embargo, se observa una diferencia significativa en los tiempos de ejecución. La fuerza bruta mediante backtracking tarda más, ya que explora una gran cantidad de combinaciones posibles. En contraste, la programación dinámica exacta obtiene la misma solución en un tiempo considerablemente menor.

Esto demuestra que la programación dinámica representa una mejora efectiva frente al enfoque exhaustivo.

7.2. Comparación entre enfoques

Criterio	Fuerza Bruta / Backtracking	Programación Dinámica
Encuentra solución óptima	Sí	Sí
Complejidad	Exponencial	Pseudo-polinomial
Tiempo de ejecución	Alto	Bajo
Escalabilidad	Baja	Media
Uso recomendado	Instancias pequeñas	Instancias medianas

Cuadro 3: Comparación entre enfoques algorítmicos.

7.3. Número de iteraciones estimadas

7.3.1. Fuerza Bruta / Backtracking

En la solución base, cada trabajo puede estar en dos estados: seleccionado o no seleccionado. Por lo tanto, el número de subconjuntos posibles es:

$$2^n$$

Para el caso estudiado:

$$2^{15} = 32768$$

Por lo tanto, el algoritmo de fuerza bruta/backtracking evalúa hasta 32768 subconjuntos.

7.3.2. Programación Dinámica

En la solución mediante programación dinámica se utiliza una tabla de tamaño:

$$(n + 1)(H + 1)$$

Donde:

$$n = 15$$

$$H = \max(d_j) = 12$$

Entonces:

$$(15 + 1)(12 + 1) = 16 \cdot 13 = 208$$

Por lo tanto, la tabla de programación dinámica contiene 208 estados.

Además, el llenado principal de la tabla realiza aproximadamente:

$$n(H + 1)$$

iteraciones principales. Para el caso estudiado:

$$15(12 + 1) = 15 \cdot 13 = 195$$

7.4. Comparación directa

Para el caso experimental utilizado, se tiene un total de $n = 15$ trabajos y un horizonte de planificación $H = 12$, correspondiente al mayor deadline del conjunto de datos.

Método	Fórmula	Caso	Cantidad aproximada
Fuerza Bruta / Backtracking	2^n	2^{15}	32768 subconjuntos
Programación Dinámica	$(n + 1)(H + 1)$	$16 \cdot 13$	208 estados
Llenado DP	$n(H + 1)$	$15 \cdot 13$	195 iteraciones

Cuadro 4: Comparación de cantidad de estados o iteraciones.

7.5. Comparación entre lenguajes

Los tres lenguajes obtuvieron la misma solución óptima, lo que confirma la consistencia de las implementaciones. Las diferencias de tiempo pueden explicarse por la forma de implementación, el manejo interno de memoria, la precisión del sistema de medición, el entorno de ejecución y los costos propios de cada lenguaje.

En este caso, la mejora más significativa no proviene del lenguaje utilizado, sino del enfoque algorítmico aplicado.

7.6. Escalabilidad

La solución por fuerza bruta no escala adecuadamente, ya que el número de subconjuntos crece exponencialmente:

$$2^n$$

Para 15 trabajos, ya se deben evaluar miles de combinaciones. Si se aumenta el número de trabajos, el tiempo de ejecución crece rápidamente.

La programación dinámica, en cambio, depende de:

$$n \cdot H$$

por lo que resulta mucho más eficiente para instancias donde el horizonte de planificación no es demasiado grande.

8. Conclusiones

El desarrollo del problema de *Job Sequencing* con duraciones variables permitió comprender en profundidad las características de los problemas NP-completos, especialmente en relación con la dificultad de encontrar soluciones óptimas en tiempos computacionales razonables.

A través del modelamiento matemático, se logró representar formalmente el problema mediante variables de decisión, restricciones y una función objetivo orientada a la maximización de la ganancia. Esto facilitó la comprensión de cómo problemas reales pueden ser abordados desde una perspectiva de optimización.

En cuanto a la implementación, se analizaron dos enfoques principales. El algoritmo de fuerza bruta/backtracking permitió encontrar soluciones óptimas, evidenciando el crecimiento exponencial del espacio de búsqueda. Por otro lado, la programación dinámica exacta permitió obtener la misma solución óptima en tiempos significativamente menores.

El análisis experimental permitió comparar el comportamiento de los distintos métodos, evidenciando que, si bien los algoritmos exactos aseguran soluciones óptimas, presentan diferencias importantes en términos de escalabilidad y tiempo de ejecución.

Además, la implementación en distintos lenguajes de programación permitió observar diferencias prácticas en términos de rendimiento y facilidad de desarrollo, aportando una visión más completa del problema desde el punto de vista computacional.

En conclusión, el problema de *Job Sequencing* con duraciones variables representa de forma clara el equilibrio entre optimalidad y eficiencia en la resolución de problemas complejos.